RAG-Based Customer Support Assistant

---

1. Introduction

1.1 Purpose

Customer support teams are overwhelmed by repetitive queries that can be answered from existing documentation (return policies, cancellation rules, product manuals). Traditional keyword-search bots fail on paraphrased questions. Fine-tuned models go stale as policies change. A RAG system solves this by retrieving from a live, updatable document corpus at query time — no retraining needed.

1.2 Scope

The system:

- Processes a PDF-based knowledge base
- Retrieves relevant information using embeddings
- Generates contextual answers using LLMs
- Routes queries using a graph-based workflow
- Supports Human-in-the-Loop (HITL) escalation

1.3 Definitions

- **RAG**: Retrieval-Augmented Generation
- **LLM**: Large Language Model
- **HITL**: Human-in-the-Loop
- **Vector Database**: Stores embeddings for similarity search

---

2. System Overview

The system is designed as an intelligent customer support assistant that combines retrieval mechanisms with generative AI to answer user queries. It ensures improved accuracy by grounding responses in a knowledge base.

---

3. System Architecture

3.1 Architecture Description

The system consists of the following layers:

1. User Interface (Streamlit)
2. API Layer (FastAPI)
3. Workflow Orchestration (LangGraph)
4. Retrieval Layer (ChromaDB + BM25)
5. LLM Processing Layer (Groq/Gemini)
6. HITL Escalation Module

3.2 Architecture

```
User Interface (Streamlit)
        ↓
FastAPI Backend
        ↓
LangGraph Workflow Engine
        ↓
Intent Classification → Agent Routing
        ↓
Hybrid Retrieval (Vector + BM25)
        ↓
LLM Processing (Groq / Gemini)
        ↓
Response Evaluation
        ↓
HITL Escalation (if needed)
        ↓
Final Response
```

---

4. Component Description

4.1 Document Loader

Loads PDF documents using a document loader and extracts textual data.

4.2 Chunking Strategy

- Chunk Size: 800 characters
- Overlap: 100 characters
- Ensures contextual continuity

4.3 Embedding Model

- HuggingFace embeddings (`all-mpnet-base-v2`)
- Converts text into vector representations

4.4 Vector Store

- ChromaDB
- Stores embeddings for semantic retrieval

4.5 Retriever

- Hybrid retrieval approach:
  - Semantic search (vector similarity)
  - Keyword search (BM25)

4.6 LLM

- Groq (primary)
- Gemini (fallback)

4.7 Workflow Engine

- LangGraph manages execution flow
- Implements conditional routing

4.8 HITL Module

Handles escalation to human agents when:

- Confidence is low
- Query is complex

---

5. Data Flow

5.1 Ingestion Flow

1. Load PDF
2. Split into chunks
3. Generate embeddings
4. Store in vector database

5.2 Query Flow

1. User submits query
2. Retrieve relevant chunks
3. Classify intent
4. Route to appropriate agent
5. Generate response
6. Evaluate confidence
7. Escalate if required
8. Return response

---

6. Technology Stack

Component	Technology
UI	Streamlit
Backend	FastAPI
Workflow	LangGraph
Vector DB	ChromaDB
Embeddings	HuggingFace
LLM	Groq / Gemini

---

7. Scalability Considerations

- Efficient chunking for large documents
- Horizontal scaling of API
- Low-latency inference using Groq
- Future support for distributed vector storage